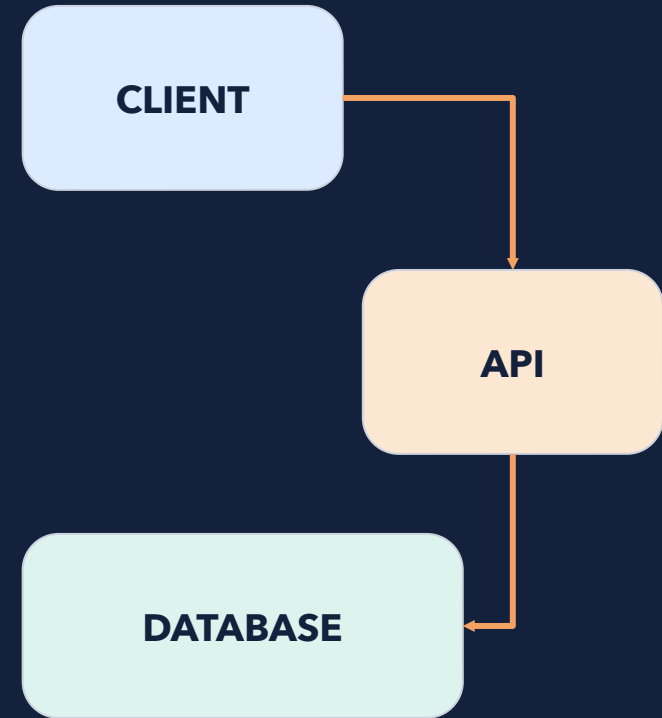


How Applications Use Databases

From one user action to a stored record, then from tables to relational reasoning.



Every familiar app is a database interface

The screen is only the visible edge of a larger system.

Music service

Artists · tracks · playlists
listening history · subscriptions

Typical question:
"Which tracks are in this playlist, in
order?"

Online store

Customers · products · orders
stock levels · payments

Typical question:
"Can this item be sold without
overselling inventory?"

City service

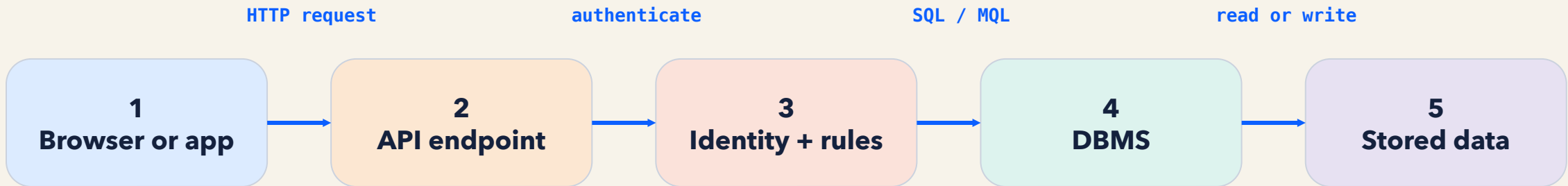
Residents · requests · locations
status changes · staff assignments

Typical question:
"Which open requests are assigned
to this district?"

Different products, same engineering problem: preserve meaning while many people read and change shared data.

One click crosses several system boundaries

A slow or failed Save can originate in more than one layer.



REQUEST

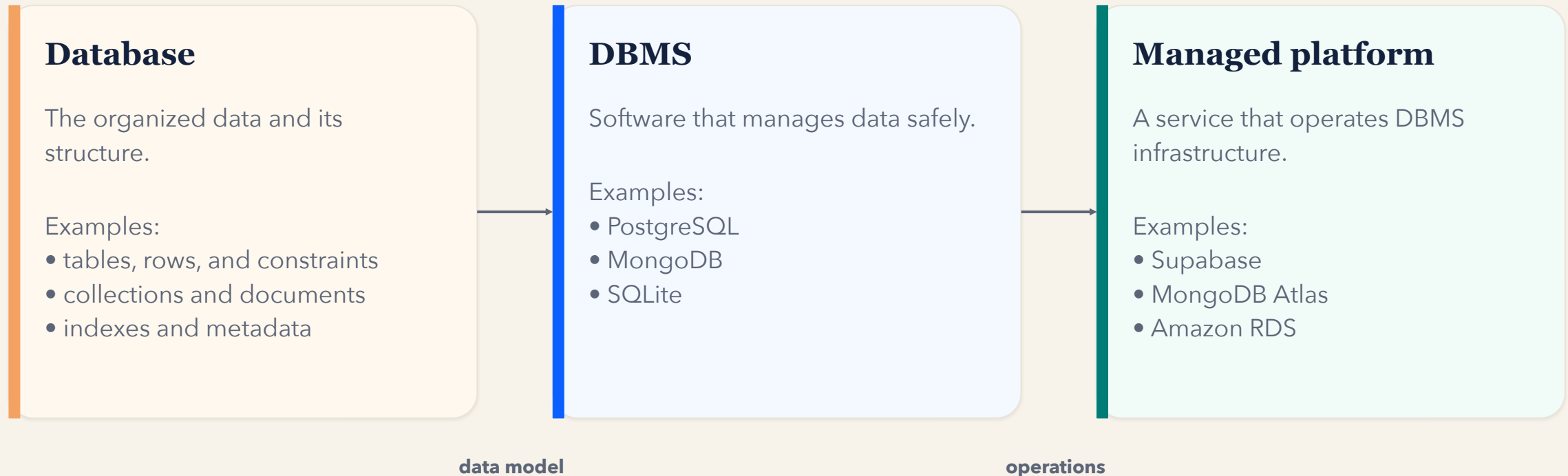
```
POST /api/tickets
{ "title": "Broken hydrant",
  "district": 5 }
```

DATABASE COMMAND

```
INSERT INTO tickets (title, district)
VALUES ('Broken hydrant', 5);
```

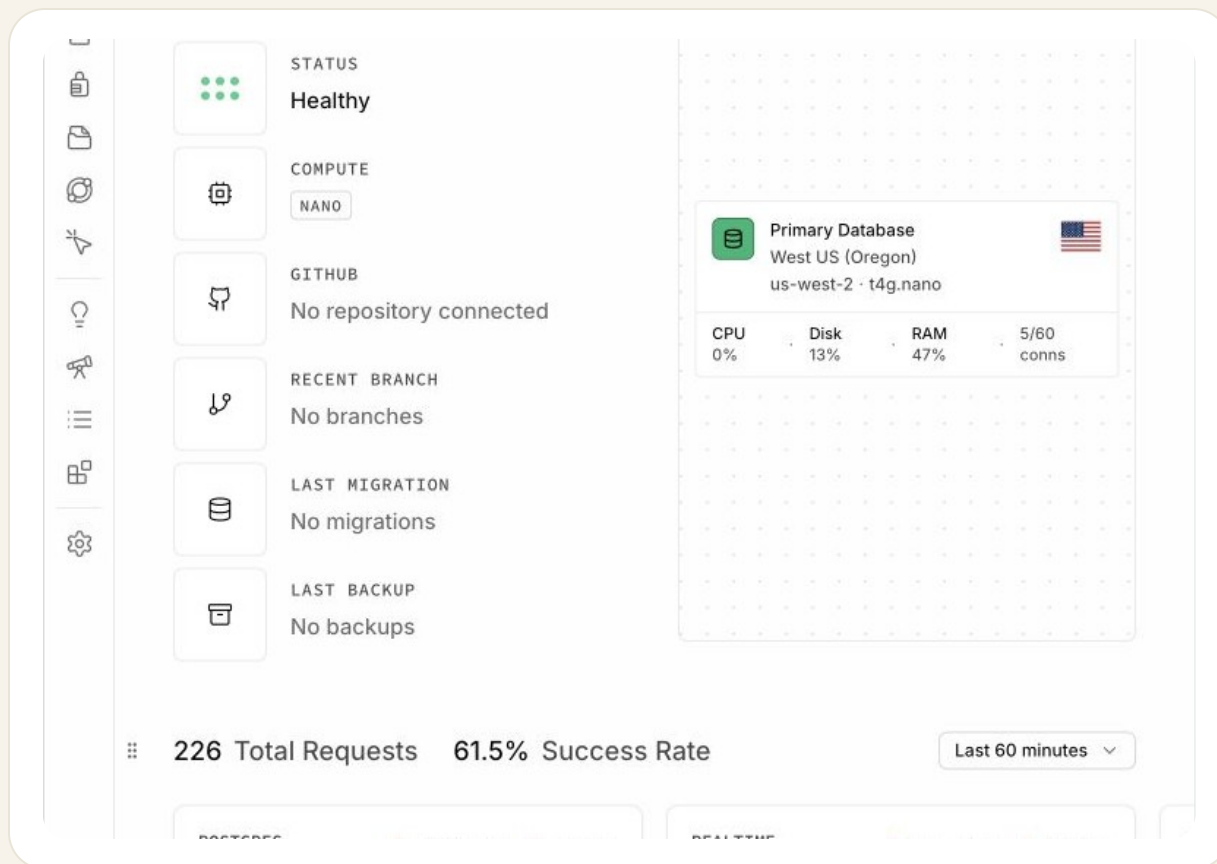
Database, DBMS, and cloud platform are not synonyms

Separating the layers makes architecture and troubleshooting clearer.



Supabase is a platform built around PostgreSQL

The dashboard exposes projects and services; PostgreSQL remains the database engine.



What the platform adds

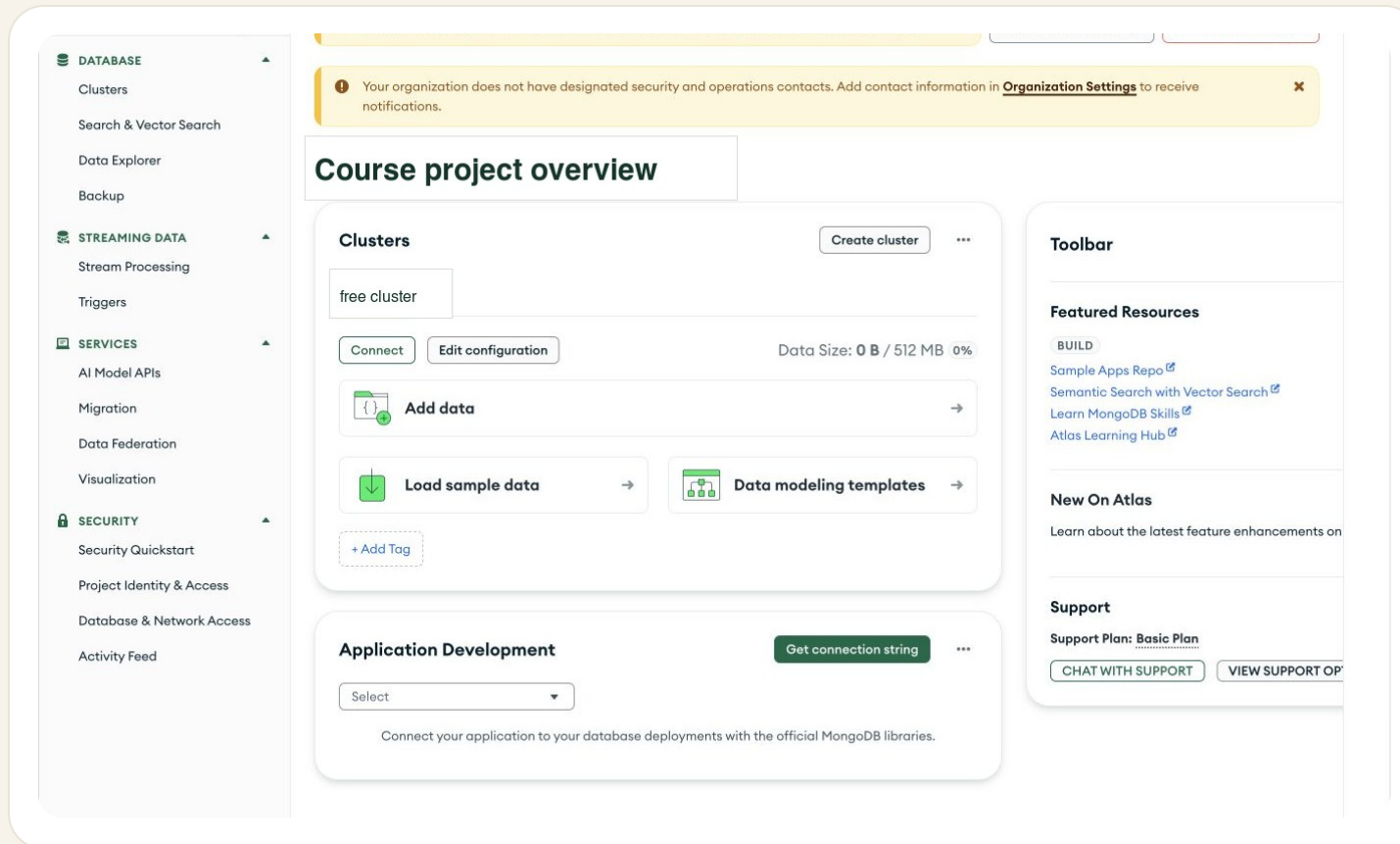
Dashboard · managed infrastructure · API services · authentication · logs · configuration

What PostgreSQL still does

Tables · SQL · constraints · transactions · query planning · indexes · roles

MongoDB Atlas operates MongoDB in the cloud

Projects, clusters, users, network rules, and data tools surround the document database.



Atlas layer

Project · cluster · network access ·
database users · logs

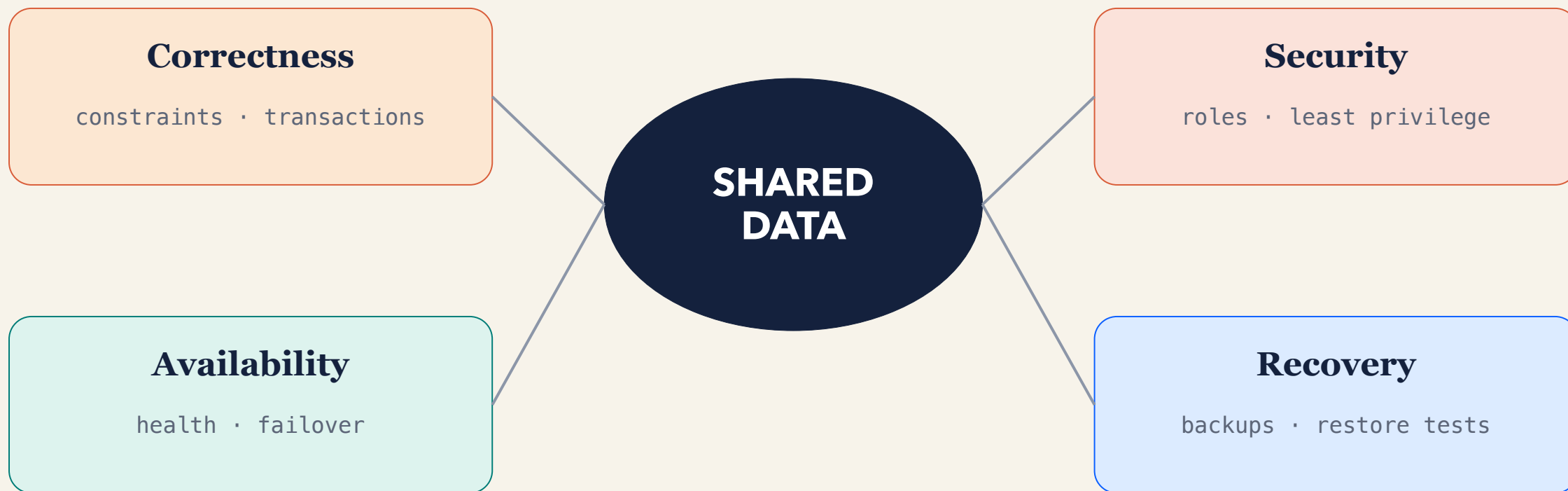
MongoDB layer

Databases · collections · BSON
documents · queries · indexes

Free-tier clusters may pause after inactivity
and can be resumed from the project.

Database administration is controlled change

The job is to keep shared data correct, available, secure, and recoverable.



A symptom is not yet a cause

Trace the request and test each boundary with the smallest useful observation.

Observed symptom

The ticket form says “Saved,”
but the new ticket is missing.

ONE OBSERVATION

```
HTTP 201 Created  
{ "id": 184, "status": "open" }
```

1

Client

Did the browser send the request?

2

API

What status and body came back?

3

Database

Did the command commit?

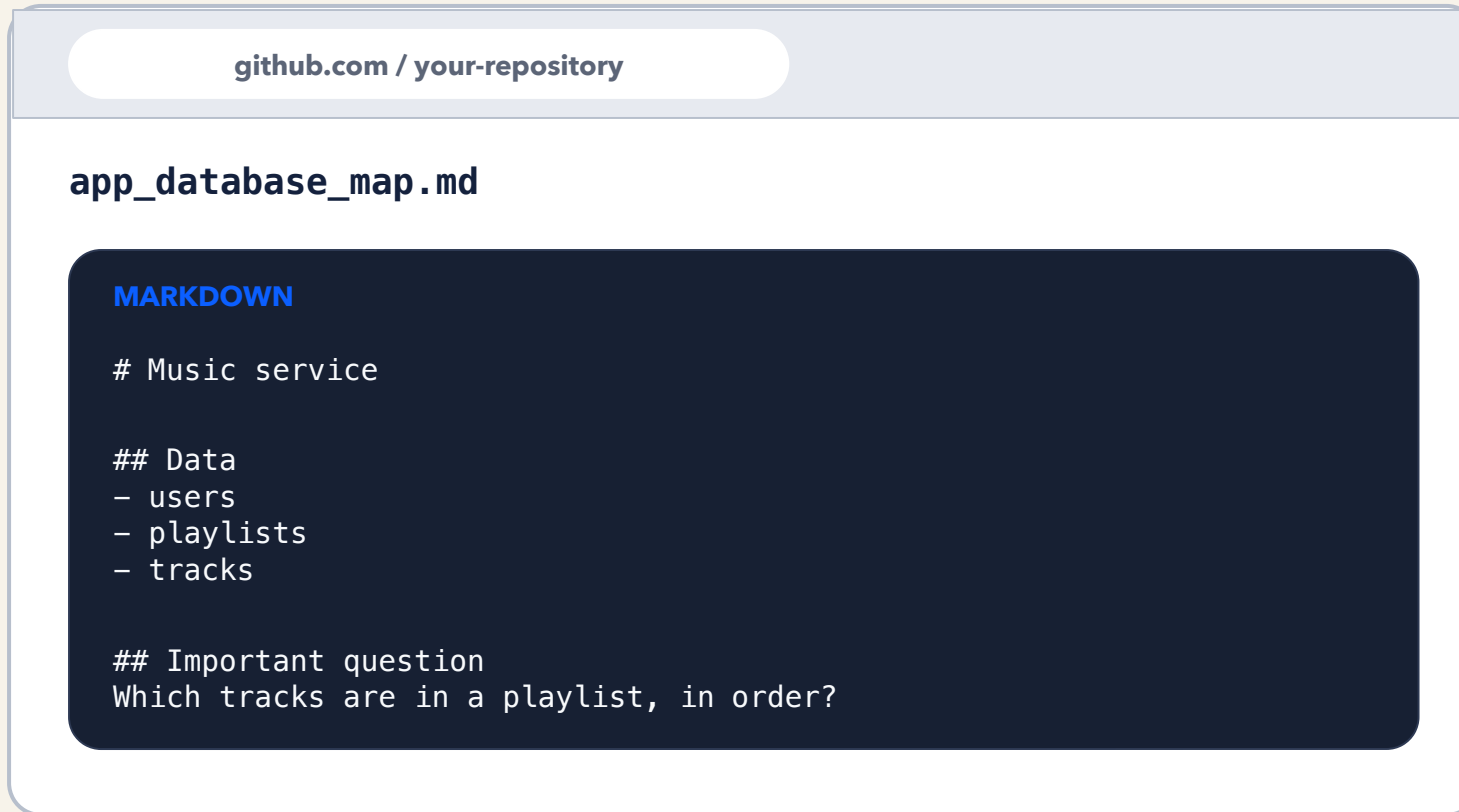
4

Read path

Is the later query filtering it out?

GitHub's browser editor is enough for today

Write Markdown in the repository, preview it, and submit the file URL.



- 1 **Open the assigned repository**
- 2 Press . or choose Edit
- 3 Create the requested file
- 4 Preview, commit, submit its URL

Map one application to its database

Individual work. One Markdown file. One submitted URL.

LAB 1

Choose a music service, store, game, or public service.

CREATE THIS FILE

`week_01/app_database_map.md`

Submit the GitHub URL for that file before leaving class.

1

Name the important data

List the entities or facts the application must remember.

2

Name the important questions

Write three questions the application must answer.

3

Trace one request

Describe the client → API → identity → DBMS → data path.

4

Sketch two data shapes

Show one small table idea and one small JSON document idea.

The Relational Model

A precise language for tables, questions, and transformations

Tables are not spreadsheets. They represent relations governed by structure and rules.

RELATIONAL ALGEBRA

```
 $\sigma$  status = 'open' (tickets)  
 $\pi$  ticket_id, title ( ... )
```

A relation represents a set of structured facts

This tickets relation has four attributes and three tuples.

ticket_id	title	status	requester_id
101	Broken hydrant	open	7
102	Streetlight out	closed	3
103	Missed pickup	open	7

schema

attribute names + domains

tuple

one complete row of values

key

stable identity

Schema and instance answer different questions

The schema describes allowed structure; the instance is the data present now.

Schema

```
tickets(  
  ticket_id integer,  
  title text,  
  status text,  
  requester_id integer  
)
```

Changes deliberately and infrequently.

Instance

The tuples stored at one moment.

At 10:00: 3 tickets

At 10:05: 4 tickets

Changes whenever rows are inserted, updated, or deleted.

Domain example

`status ∈ {'open', 'closed'}`

A domain says which kind of value is valid for an attribute.

Keys connect facts without repeating entire records

A foreign key stores the identifier of a tuple in another relation.

users

user_id	name
3	Mina
7	Luis
9	Jo



FK → PK

tickets

ticket_id	title	requester_id
101	Broken hydrant	7
102	Streetlight out	3
103	Missed pickup	7

ticket 101 belongs to user 7, without storing Luis's name in every ticket row

Selection keeps tuples that satisfy a condition

The symbol sigma, σ , corresponds closely to a SQL WHERE condition.

ticket_id	title	status
101	Broken hydrant	open
102	Streetlight out	closed
103	Missed pickup	open

 σ **ALGEBRA**

σ status = 'open' (tickets)

SQL

```
SELECT *  
FROM tickets  
WHERE status = 'open';
```

Projection keeps selected attributes

The symbol π , π , corresponds to the output list after SQL SELECT.

ticket_id	title	status	requester_id
101	Broken hydrant	open	7
102	Streetlight out	closed	3
103	Missed pickup	open	7

 π

ALGEBRA

 π ticket_id, title (tickets)

SQL

```
SELECT ticket_id, title  
FROM tickets;
```

Output schema: (ticket_id, title)

Product pairs rows; join keeps meaningful matches

A join is not merely placing two tables side by side.

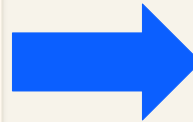
Cartesian product $R \times S$

Pairs every tuple in R with every tuple in S.

If $|R| = 3$ and $|S| = 4$, then:

$$|R \times S| = 12$$

Useful as a definition; usually too broad as a result.



condition

Equi-join $R \bowtie S$

Keeps only pairs satisfying an equality condition.

`tickets.requester_id`
`= users.user_id`

Reconstructs related facts when a query needs them together.

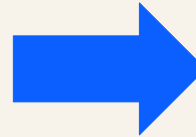
Join cardinality depends on the data and the condition, not only on input sizes.

Relational reasoning explains what the SQL is doing

Read the query as a sequence of transformations, not a chant of keywords.

RELATIONAL ALGEBRA

```
π title, name (  
  σ status = 'open' (  
    tickets ⋈ requester_id = user_id  
    users  
  )  
)
```



SQL

```
SELECT t.title, u.name  
FROM tickets AS t  
JOIN users AS u  
  ON t.requester_id = u.user_id  
WHERE t.status = 'open';
```

1 join related facts

2 select open tickets

3 project title + name

Translate questions into relational operations and SQL

Individual work. One Markdown file. No database connection is required.

LAB 2

Use the users and tickets relations from today.

CREATE THIS FILE

`week_01/relational_reasoning.md`

Part A • Read

For three given expressions, state the output attributes and which tuples would appear.

Part B • Write

For two plain-language questions, write relational algebra and corresponding SQL.

Submit the file URL before leaving class.

A database course starts with systems and meaning

Next: SQL retrieval, joins, grouping, subqueries, and window functions.

Trace

client → API → identity → DBMS → data

Distinguish

database ≠ DBMS ≠ managed platform

Model

relations, attributes, tuples, domains, keys

Transform

σ selection · π projection · $\bowtie$ join

Before Week 2: read Chapter 2, especially §§ 2.5-2.8, and bring the SQL Query Ladder lab.